Automation and rollout plan

Everything marked **built** runs in this repository today; **planned** does not exist yet. Projections are **estimates** from `model/business_case.xlsx` (inputs tagged MEASURED / PUBLIC / ASSUMED).

1. The automated step, to production standard

The automated step: *a document arrives; it is read, classified, checked, filed into the right folder and published to the knowledge base, or it goes to a reviewer with the reasons.*

Requirement	State	How
Safe to re-run	Built	Content-hash document id; a done document is skipped unless forced; per-page OCR and extraction cache, so a re-run resumes and costs nothing for finished pages; re-filing replaces the earlier copy and prunes empty folders; KB publishing removes the old copy first
Monitored	Built	`docpipe/monitor.py` reads the audit log and the review queue: documents, auto-file and review rates, validator failure rate, schema-invalid replies, model errors, p50/p95 latency, cost per document, KB publish errors, auto-filed-then-corrected, kill switch
Alerts	Built	Thresholds in `config/pipeline.yaml` (`monitoring`); an alert is written once when it starts and once when it resolves, to `alerts.jsonl`, the audit log, the UI banner, and a webhook (Teams/Slack) if configured; the web server re-checks every 10 minutes, `docpipe monitor` on demand
Kill switch	Built	`docpipe kill on` or the UI toggle: nothing is auto-filed, everything goes to review; logged with the actor
Rollback	Built	Every replaced filed version is kept in `_history/<doc>/<n>/`; `docpipe rollback <doc> [--version N] --actor NAME`, or the UI "Versions" tab, restores it to its folder and re-publishes it; `docpipe rollback --release 0.3` undoes every document filed by a bad pipeline version; the rollback itself is snapshotted, so it can be undone
Human in the loop	Built	Review queue with reasons and the page image; approve, or correct one value (re-validated, re-filed, logged with the reviewer)

2. Governance: what the tool may decide on its own

- **Confidence comes from checks, not from the model.** Each failed check halves the confidence; the auto-file threshold is 0.90 (`governance.auto_file_threshold`), to be recalibrated on pilot data.
- **The review queue receives**: failed validators or evidence checks, low confidence, a document type outside the taxonomy, no readable text, model errors, and everything while the kill switch is on. Nothing in the queue is published to the knowledge base until a person approves it.
- **Audit log**: every model call (model, prompt hash, tokens, latency, raw response), check, decision, approval, correction, rollback, alert and kill-switch change, with the actor and the time.

What breaks if the tool is wrong

Failure	What catches it	If it is not caught
A digit misread on a statement	"Number printed on the page" check (text PDFs), two-pass agreement (scans), accounting identities	A wrong value in a spread; the weekly audit sample is the backstop; the "auto-filed then corrected" alarm fires when found
Wrong document type or wrong company (the bank, seller or auditor picked instead of the customer)	Fixed taxonomy; the evidence quote and company name must appear in the OCR text; customer rule in the prompt (6/6 traps handled)	Misfiled document; found by the next person who opens the folder; fix is a re-run or rollback
Model adds text that is not on the page	Not caught for documents without validators (seen once: an invented table header)	The KB transcript contains a line that is not on the page; citations still point to the right page image; audit sample
Hidden instructions in a document	Document text is data in the prompt; Q&A is read-only and cites pages (tested)	A wrong classification would go to review if the evidence check fails
Personal data published	Regex masking before the KB; restricted mapping kept local	Names are not masked (known gap, L2 planned)

Failure	What catches it	If it is not caught
Model behaviour changes silently	Pinned model name, prompt hash in every call, monitor alerts on validator failure and review-rate jumps, eval re-run before any upgrade	Drift until the next alert or audit sample
A bad release	Pipeline version stored on every result	`rollback --release` restores the previous versions

3. Before and after [measured where marked, otherwise estimate]

Per document	Before (manual)	After (DocPipe)
Statement file (spread)	60 min **[assumed; public range 30-360]**	30-60 s machine time **[measured]**; ~15 min review for the ~40% flagged; ~0.5 min audit sampling for the rest
Onboarding document	6 min **[assumed]**	13 s machine time **[measured]**; 2 min review for the ~30% flagged
Cost of the model	n/a	$0.0013 per general document, ~$0.004 per statement file **[measured, list price]**
Traceability	Depends on who keyed it	Every value and every filing decision traceable to a page, a check and a log line

Estimated monthly effect at the base volumes (600 statement files, 4,000 documents): ~1,000 manual hours become ~130, saving ~870 hours (5.5 FTE of capacity) **[estimate]**. The pilot alone (one team, a tenth of the volume) does not pay back the one-off cost: the case needs at least a region.

Projected at scale [estimate]: across all corporate and SME credit teams (5x the base volume, assumed) the same per-document economics give ~4,400 hours a month (~27 FTE) and about 11.7 bn VND net over 12 months, with run cost growing with the square root of volume. The number to replace first is the manual minutes per document: at 30 / 3 minutes the case is negative at every automation level (sensitivity table in the workbook).

4. Three-month rollout (team of about 5)

Team: product and process owner (1), engineers (2), risk and compliance partner (1, part time), an analyst from the pilot team (1, part time).

Weeks	Phase	Deliverables	Gate to proceed
1-2	Foundations	Five stakeholder interviews; timed baseline and error audit of current work; model endpoint decision (in-bank or contractual); legal review of OpenViking (AGPL) and cross-border data; thresholds and stop criteria signed by risk	Risk signs the data-handling approach; baseline recorded; business case re-run on the timed baseline
3-6	Shadow pilot (one credit team)	DocPipe runs beside manual work; nothing it produces is used; weekly audit sample; 200 real approved documents labelled as the held-out eval set; alerts wired to the team channel	Audited critical error rate on would-be auto-filed documents at or below the manual rate; review rate under 50%
7-10	A/B in assist mode	Batches randomised by day: half start from DocPipe output, half manual; review queue live; Q&A on approved documents; weekly metrics review	Handling time down at least 20% vs control; no unresolved audit finding
11-13	Harden and scale	Second team onboards by configuration only; rollback drill; risk walkthrough of the audit log; scale-up memo with measured numbers	Second team live without code changes; go / no-go for the region

Key risks and how they are reduced

Risk	Early signal	Mitigation
A wrong value passes every check	Audit sample finds an error in an auto-filed document; "auto-filed then corrected" alarm	Add the missing check; lower the auto-file share until it holds
Real scans are harder than the test set	Week-3 eval on 200 real documents	Thresholds set on real data; more goes to review until it holds

Risk	Early signal	Mitigation
Personal data reaches a hosted model	Any non-public document in scope	In-bank endpoint is a gate; privacy ladder L1 to L3
Review burden cancels the savings	Review rate and minutes per flagged document (monitored)	Tune validators on false alarms; stop criterion
Adoption: staff keep re-keying	Low use of pre-filled output in the A/B arm	Analyst in the team; page image next to every value; fix what they flag weekly
Licence or vendor issue	Legal review	KB behind an adapter (local fallback exists); contractual model endpoint

5. Reuse by another team

Another team adopts DocPipe by configuration, not code: see `docs/adoption-guide.md`. In short: copy `config/pipeline.yaml`, set the taxonomy (one line per document type), the model endpoint, thresholds and the knowledge-base root; drop 15-20 labelled samples into an eval folder and run `evals/general_docs.py` to measure before trusting it; start the web UI. A new statement layout needs validator label patterns (`docpipe/validators.py`).

Adopting DocPipe in another team (about one day)

DocPipe is configured, not re-coded, per team. A trade-finance, retail-lending or HR team can reuse the same pipeline for its own documents by following these steps.

1. Copy and edit the configuration

```
cp config/pipeline.yaml config/tradefinance.yaml
```

Change only these sections:

Section	What to set	Example (trade finance)
`taxonomy.doc_types`	One line per document type the team files; keep `other` last	`letter_of_credit`, `bill_of_lading`, `commercial_invoice`, `packing_list`, `certificate_of_origin`, `other`
`taxonomy.statement_doc_types`	Types that must contain financial statements (empty list if none)	`[]`
`paths.out`	The team's output folder (its filing root)	`data/out_tradefinance`
`kb.openviking.root`	The team's branch of the knowledge base	`viking://resources/tradefinance`
`model.name` / `model.api_base`	The approved model endpoint (through the LiteLLM gateway)	`openai/inbank-vlm`, `http://gateway:4010/v1`
`governance.auto_file_threshold`	Start strict (0.95) and lower it only with eval evidence	`0.95`
`monitoring`	Alert thresholds and the team channel webhook	`webhook_url: https://.../teams-webhook`
`pii.entities`	Identifier types to mask	add a recognizer if the team handles passport numbers

Folder layout is automatic: `{customer company}/{year}/{period or document type}/`. The classifier is told to name the bank's customer, never the bank itself; adjust the wording in `docpipe/extract.py` only if the team's "customer" means something else.

2. Measure before trusting it

1. Put 15 to 20 representative, approved sample documents in a folder, including hard ones (bad scans, the bank printed as a party, a document outside the taxonomy).
2. Write `labels.json` next to them, one entry per file (`text` is the printed text, used for the OCR error rate):

```
[{"file": "lc_0012.pdf", "doc_types": ["letter_of_credit"], "company_slug": "an-phat",
  "fiscal_year": 2025, "expected": "auto_filed", "difficulty": "clean", "note": "", "text": "..."}]
```

3. Run the evaluation into a scratch folder (it never publishes to the knowledge base):

```
uv run --group gateway python -m evals.general_docs --set data/eval/tf_set --out data/eval/tf_out --
report data/eval/tf.json
```

4. Go live only if **wrong-but-auto-filed is 0** and the review share is acceptable to the team lead. Keep the set: it is the regression test before any model or prompt change.

3. Run it

```
 scripts/start_services.ps1
uv run --group gateway python -m docpipe --config config/tradefinance.yaml serve --port 8810
```

Staff use the web UI (Inbox, Library, Review, Knowledge base, System). Operations watch the System tab or the webhook channel; the kill switch and rollback are there.

4. What needs code (rarely)

- A new financial-statement layout: label patterns in `docpipe/validators.py`.
- A new identifier to mask: a regex in `docpipe/pii.py` plus a case in `evals/pii_eval.py`.
- A different filing scheme than company / year / period: `filing.folder_parts`.

5. Ownership

Who	Owns
Team lead	Taxonomy, thresholds, the eval set, who may approve
Platform team	Gateway, model endpoint, OpenViking, upgrades (eval re-run before each)
Risk	Sign-off on thresholds, audit-sample policy, retention